

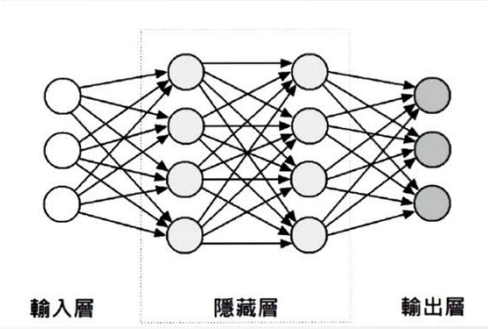
Node-RED 程式設計
AI 應用
Ch10 取得與標註YOLO 訓練資料：LabelImg

1.YOLO 物體偵測的深度學習演算法

- 官方網站：

https://pjreddie.com/darknet/yolo/

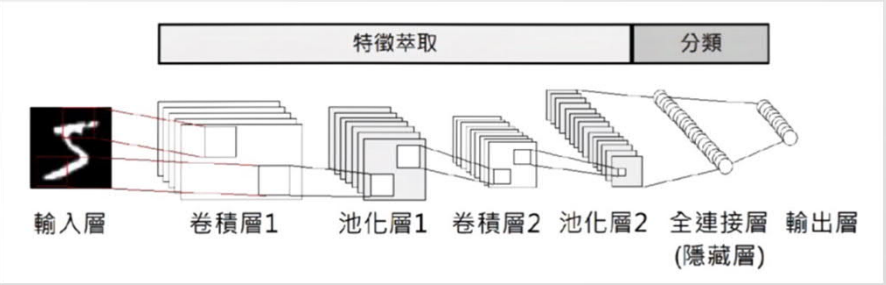
https://docs.ultralytics.com/models/yolov8



深度學習
（Deep Learning）

YOLOv8 系列的每個變體都針對其各自的任務進行了最佳化。所有變體都支援訓練、驗證、推理和匯出模式，涵蓋從開發到部署的每個階段。

模型	檔案名稱	任務	訓練	驗證	推理	出口
YOLOv8	yolov8n.pt yolov8s.pt yolov8m.pt yolov8l.pt yolov8x.pt	偵測	✅	✅	✅	✅
YOLOv8-seg	yolov8n-seg.pt yolov8s-seg.pt yolov8m-seg.pt yolov8l-seg.pt yolov8x-seg.pt	實例分割	✅	✅	✅	✅
YOLOv8-cls	yolov8n-cls.pt yolov8s-cls.pt yolov8m-cls.pt yolov8l-cls.pt yolov8x-cls.pt	分類	✅	✅	✅	✅

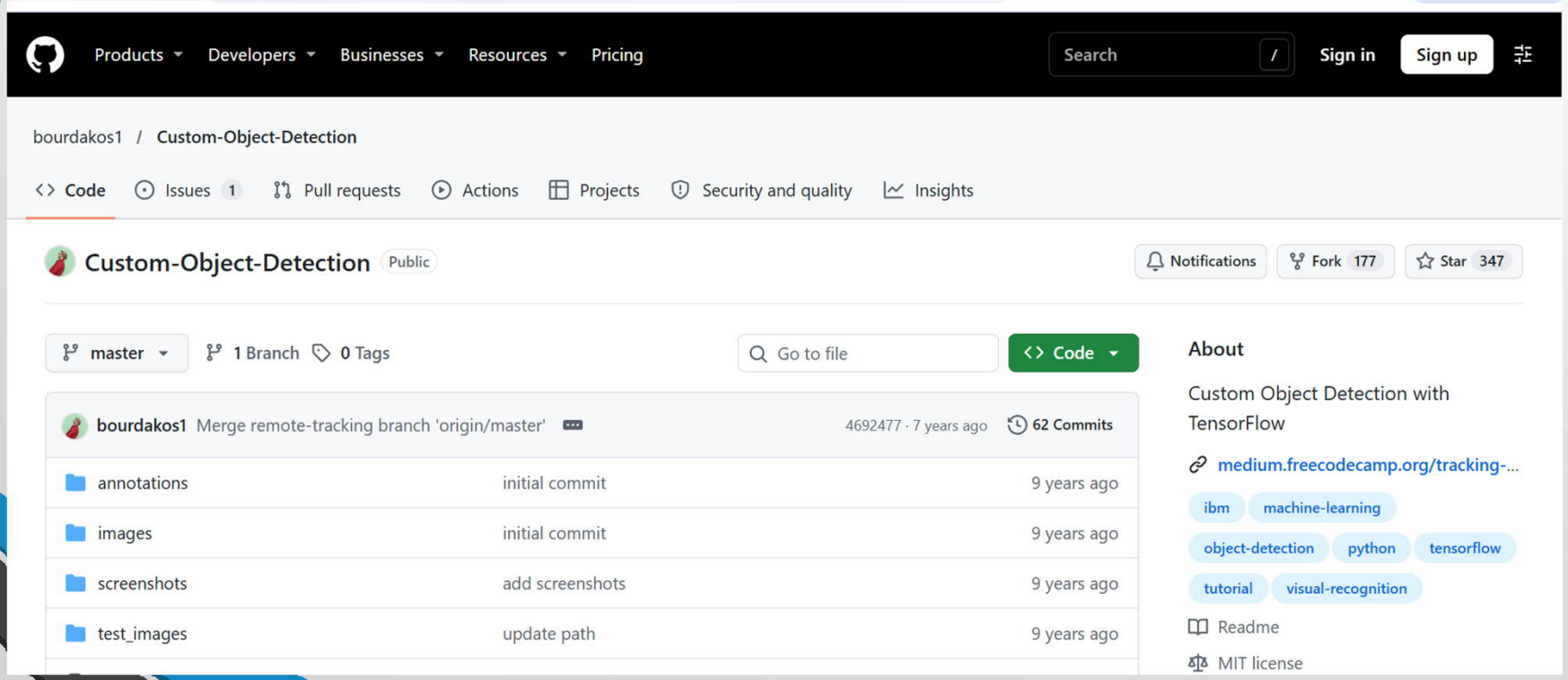


CNN (卷積神經網絡)

2. 取得YOLO模型的圖檔資料

- 如何取得圖檔來訓練AI

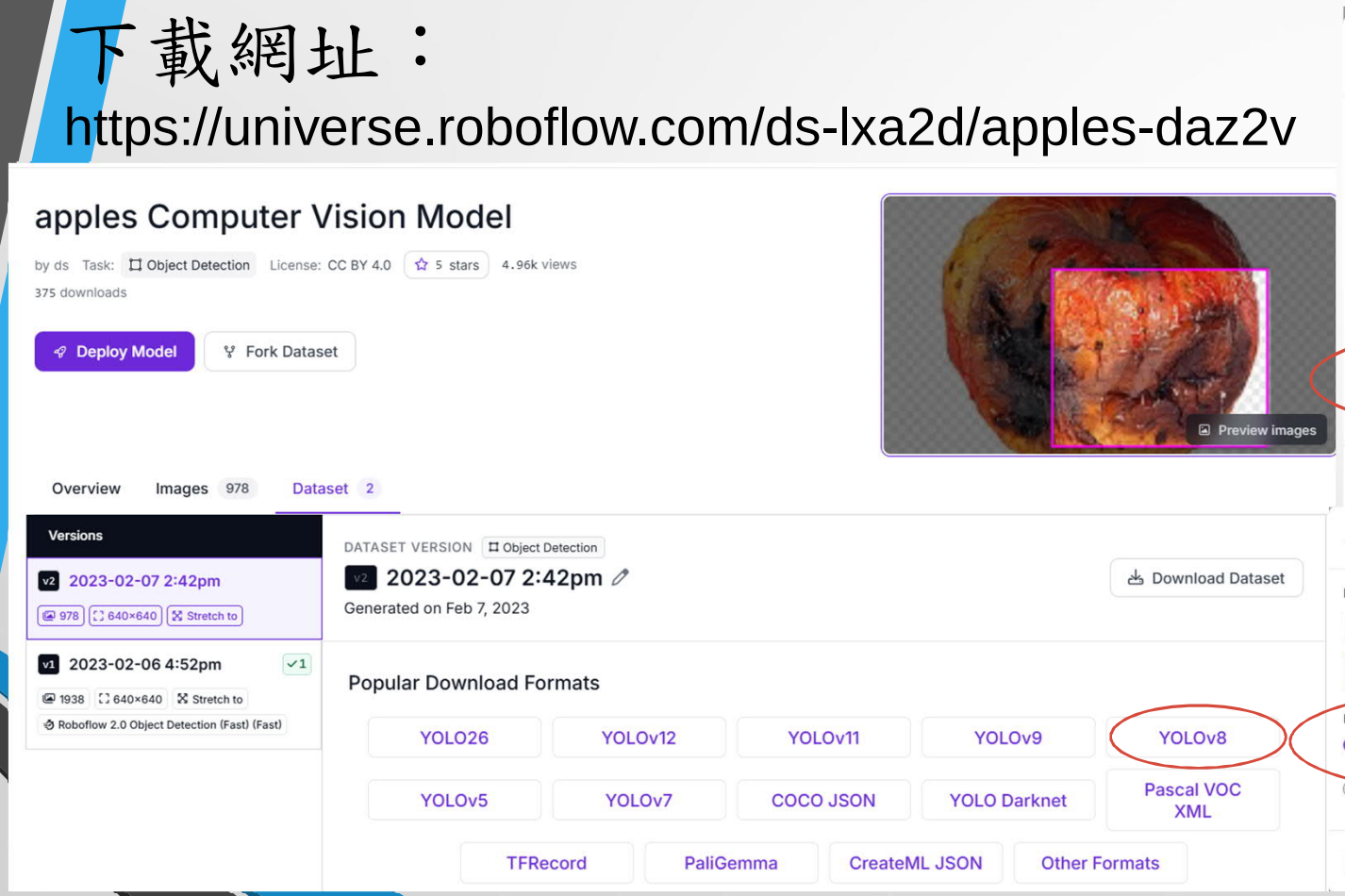
1.從GitHub網站取得訓練資料的圖檔(星際大戰電影預告)：
https://github.com/bourdakos1/Custom-Object-Detection/tree/master

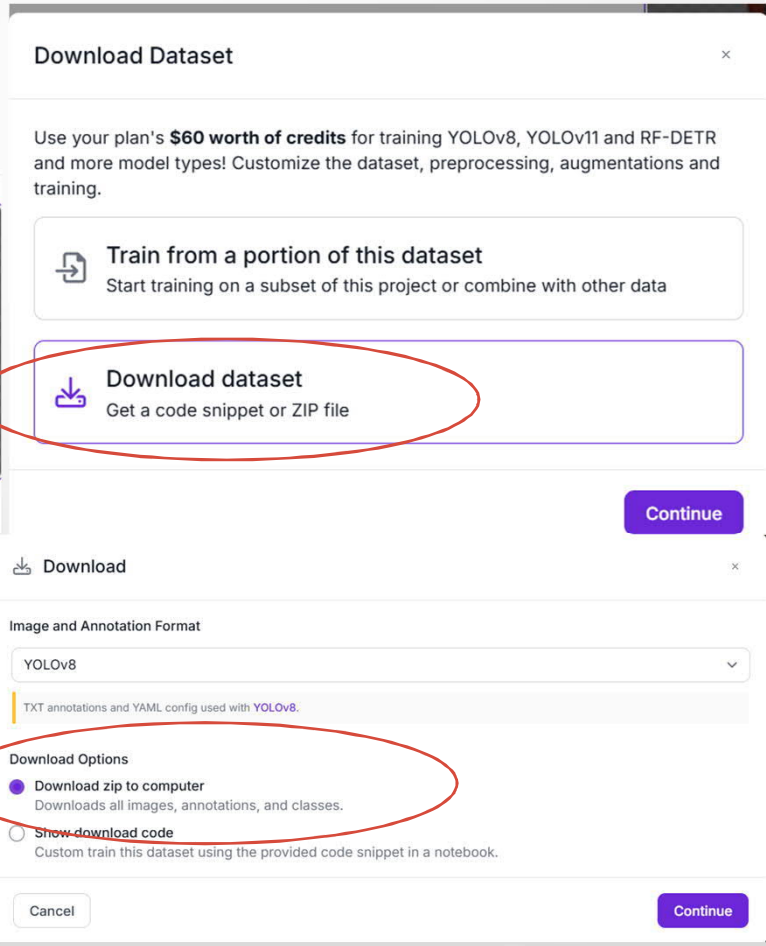


2.從Roboflow Universal 下載訓練YOLO模型的資料集

下載網址：

https://universe.roboflow.com/ds-lxa2d/apples-daz2v

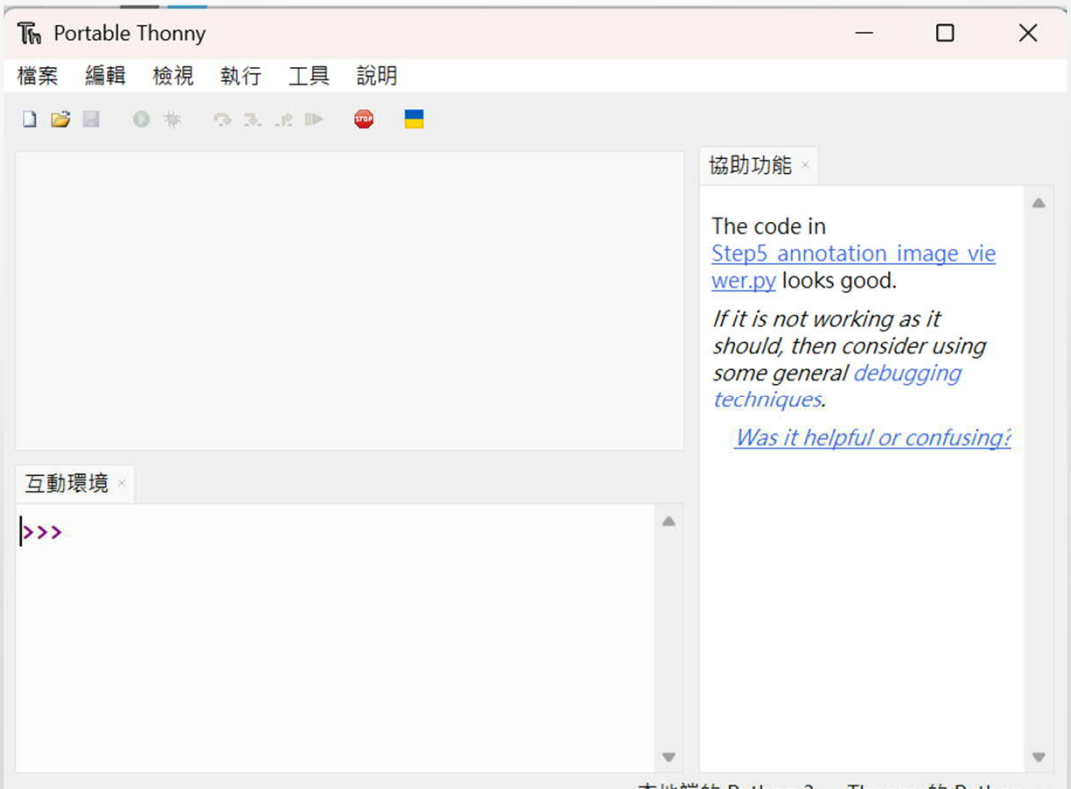




3.使用視訊影片取得訓練模型的圖檔

Step1：從網路下載MP4(使用Python程式下載)

- Python 程式簡單練習
- 使用thonny-5.0.1
- 執行 thonny.exe



➢ Python 程式簡單練習

```
print("第1個Python程式")
```

未命名 *

```
1  print("第1個Python程式")
```

互動環境

```
>>> %Run ch10-2-1.py
  第1個Python程式
>>>
```

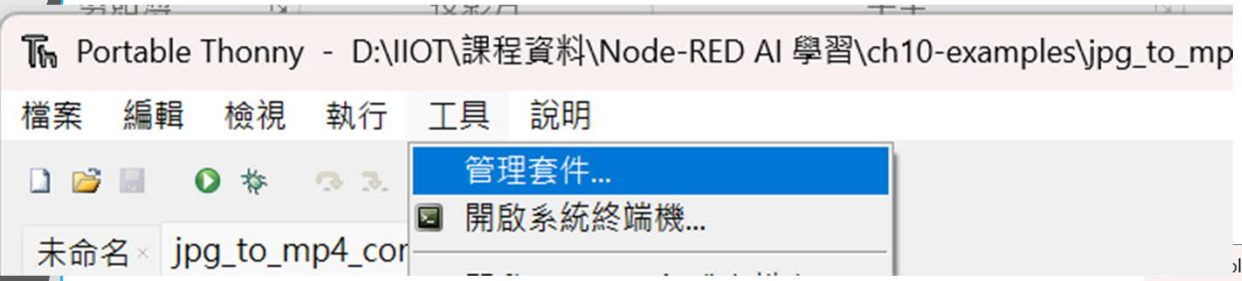
```
>>> 5+10
15
>>>
```

```
>>> num = 10
>>> print(num)
  10
>>>
```

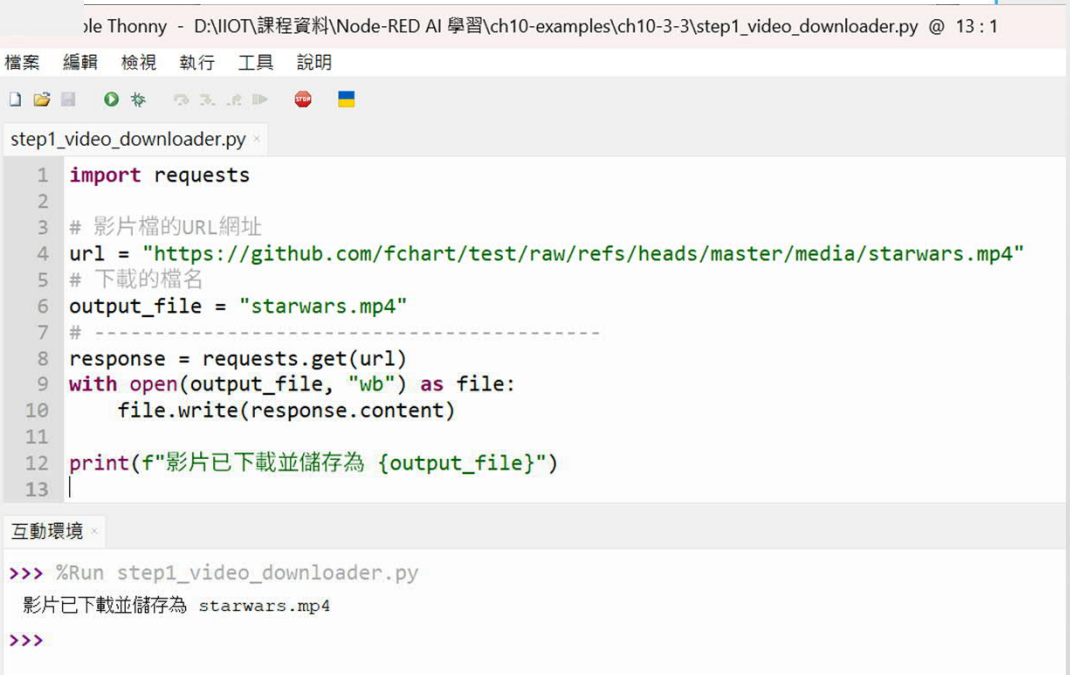
3.使用視訊影片取得訓練模型的圖檔

Step1：從網路下載MP4(使用Python程式下載)

- 先安裝 requests 網路通訊套件





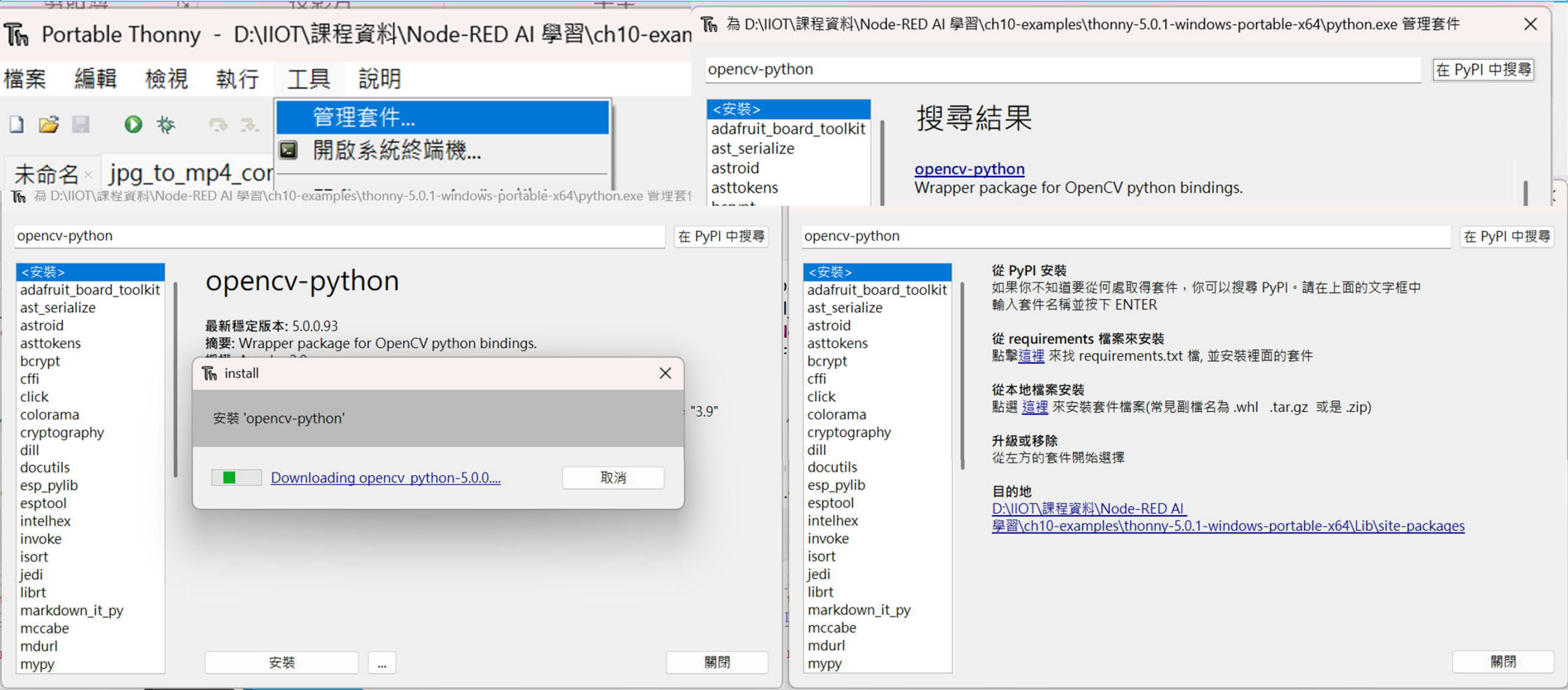


- Run

step1_video_downloader.py 下載

Step2：將MP4影片的影格輸出成JPG圖檔

- Python 環境安裝 OpenCV (opencv-python) 庫



Run step2_frame_splitter(中文字).py

step2_frame_splitter(中文字).py

```
import shutil
import urllib.request
import tempfile

# 1. 您的原始路徑與資料夾設定
base_dir = r"D:\IIOT\課程資料\Node-RED AI 學習\ch10-examples\ch10-3-3"
video_file = os.path.join(base_dir, "starwars.mp4")
output_dir = os.path.join(base_dir, "frames")

# 確保輸出目錄存在
os.makedirs(output_dir, exist_ok=True)

# 2. 自動擷截假影片與重新下載
```

影片位置

互動環境

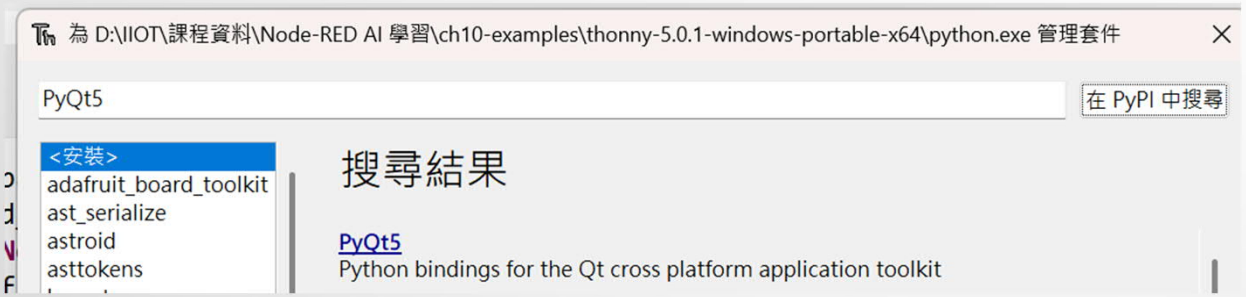
```
⚠ 發現 starwars.mp4 為無效空殼檔 (可能是下載到 GitHub 網頁代碼)！
⌛ 正在從正確的 Raw 節點重新下載真正的影片二進位檔，請稍候...
☑ 真正的高清影片重新下載完成！
🎞 開始擷取影格，這會花費幾秒鐘...
   ...已抽出 100 張圖片
   ...已抽出 200 張圖片
   ...已抽出 300 張圖片
🎉 大功告成！總共 384 個影格已成功抽出並儲存至 'D:\IIOT\課程資料\Node-RED AI 學習\ch10-examples\ch10-3-3\frames'
>>>
```

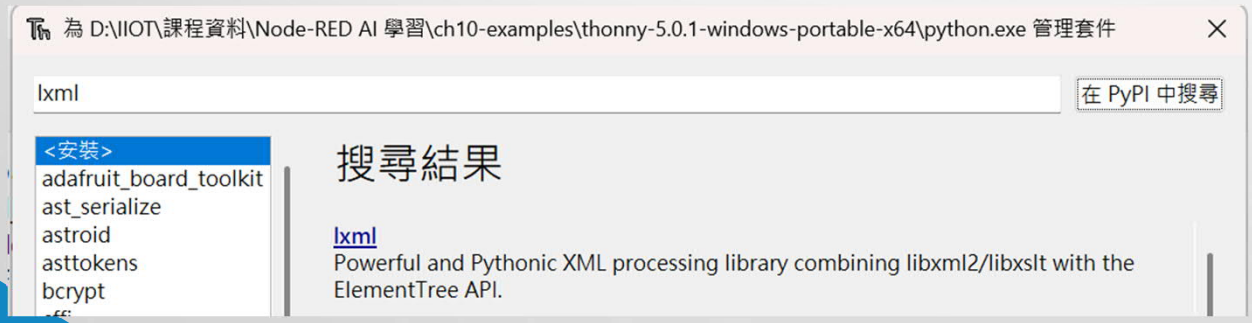
輸出成JPG圖檔

3. 使用Labellmg 標註圖檔建立訓練資料

1.認識Labellmg 與 YOLO 標註檔

- Python 環境安裝 PyQt5 & lxml庫





➢ 1.設定LabelImg 預分類資料 Step1_labelimg_preclassifier.py

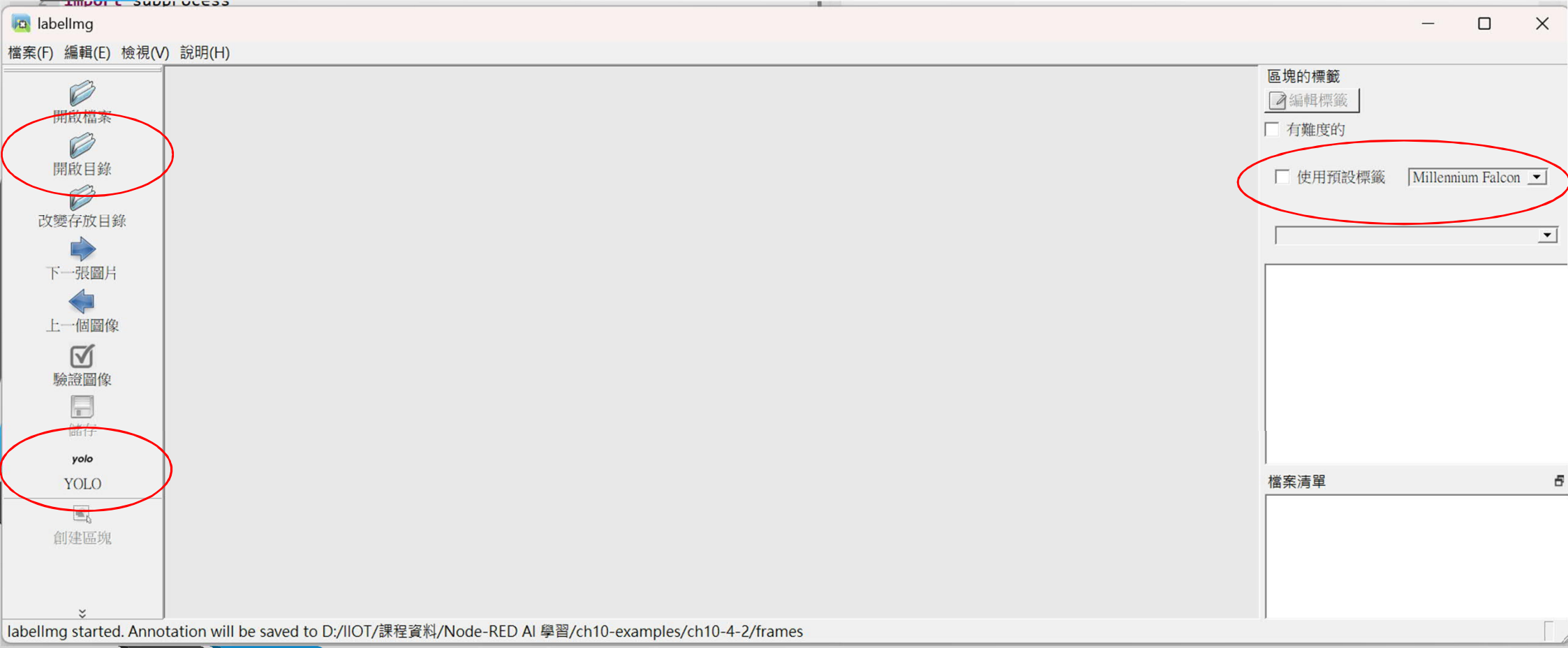
Step1_labelimg_preclassifier.py

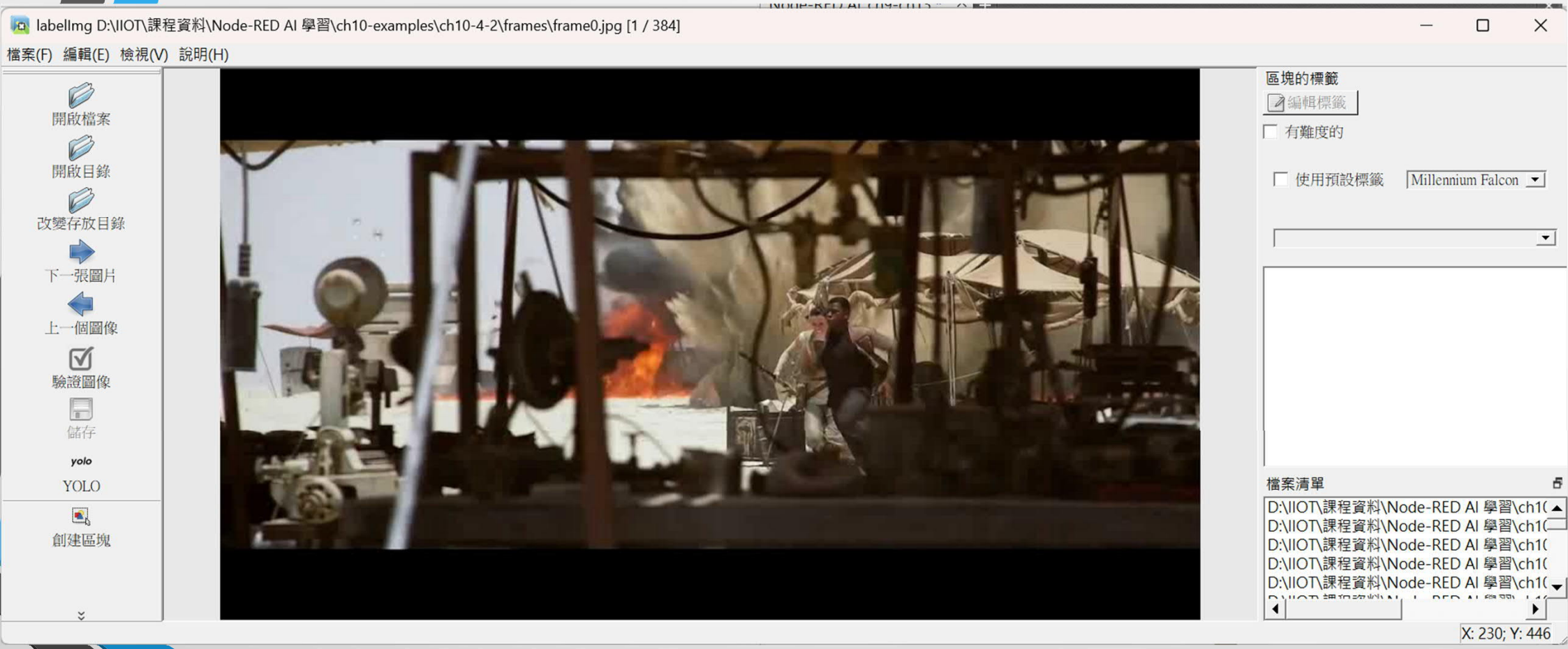
```
1  # 定義分類串列
2  class_list = ["Millennium Falcon", "Tie Fighter"]
3  # 定義要寫入的檔案路徑
4  file_path = "labelImg/data/predefined_classes.txt"
5  # ----------------------------------------------
```

predefined_classes.txt

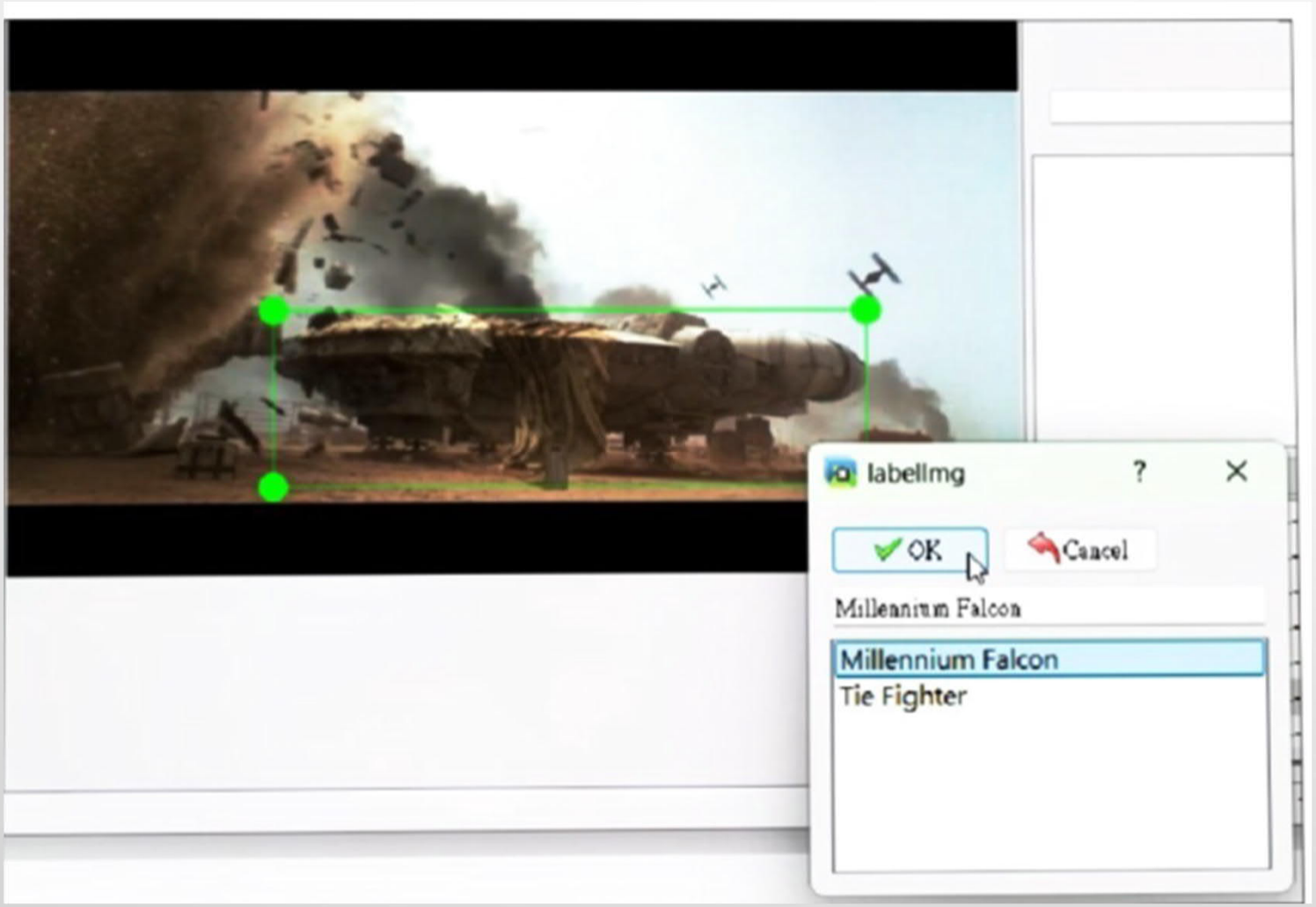
```
1  Millennium Falcon
2  Tie Fighter
3
```

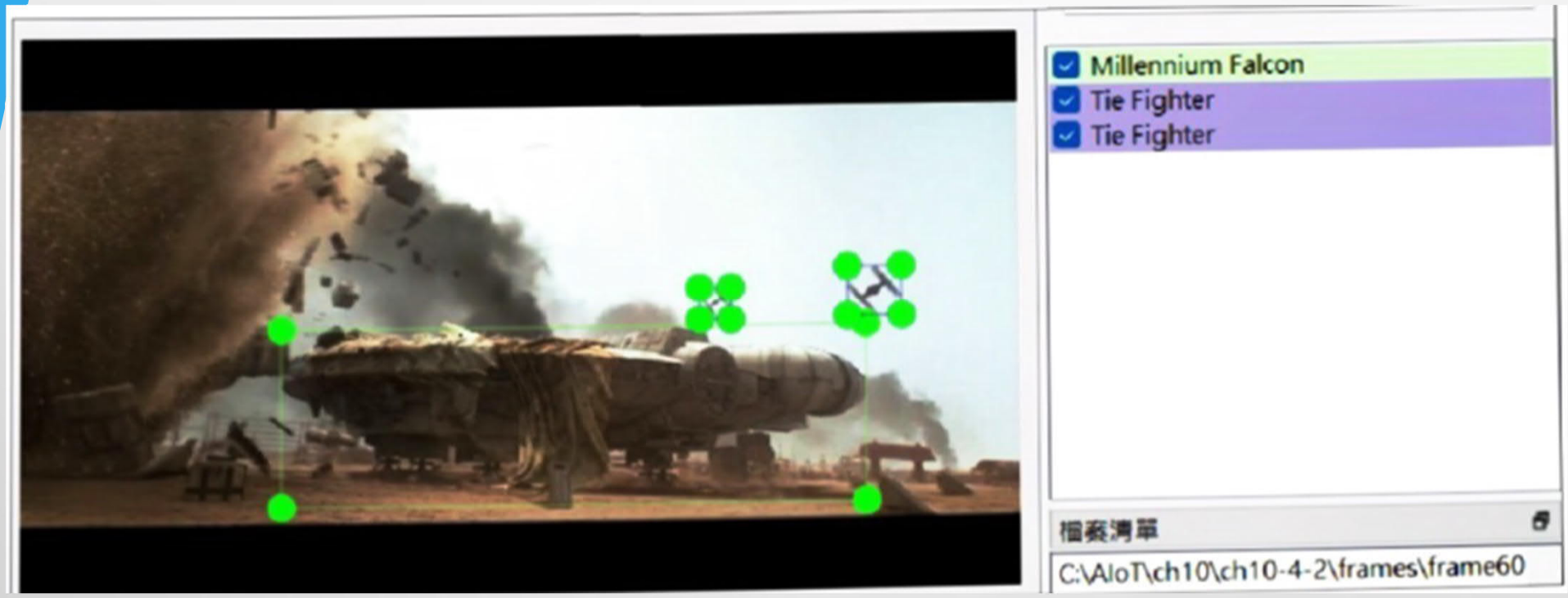
➢ 2.啟動和使用Labellmg 影像標註工具 Step2_labelimg_launcher_(中文檔名).py




labelImg D:\IIOT\課程資料\Node-RED AI 學習\ch10-examples\ch10-4-2\frames\frame0.jpg [1 / 384]
檔案(F) 編輯(E) 檢視(V) 說明(H)
開啟檔案
開啟目錄
改變存放目錄
下一張圖片
上一個圖像
驗證圖像
儲存
YOLO
創建區塊
區塊的標籤
編輯標籤
有難度的
使用預設標籤
Millennium Falcon
檔案清單
D:\IIOT\課程資料\Node-RED AI 學習\ch1(
D:\IIOT\課程資料\Node-RED AI 學習\ch1(
D:\IIOT\課程資料\Node-RED AI 學習\ch1(
D:\IIOT\課程資料\Node-RED AI 學習\ch1(
D:\IIOT\課程資料\Node-RED AI 學習\ch1(
X: 230; Y: 446


labelImg C:\AIoT\ch10\ch10-4-2\frames\frame65.jpg [66 / 384]
檔案(F)
編輯(E)
檢視(V)
說明(H)
開啟檔案
開啟目錄
改變存放目錄
下一張圖片
上一個圖像
驗證圖像
儲存
YOLO
創建區塊
區塊的標籤
編輯標籤
有難度的
使用預設標籤
Millennium Falcon
檔案清單
C:\AIoT\ch10\ch10-4-2\frames\frame60
C:\AIoT\ch10\ch10-4-2\frames\frame61
C:\AIoT\ch10\ch10-4-2\frames\frame62
C:\AIoT\ch10\ch10-4-2\frames\frame63
C:\AIoT\ch10\ch10-4-2\frames\frame64
C:\AIoT\ch10\ch10-4-2\frames\frame65
C:\AIoT\ch10\ch10-4-2\frames\frame66
Width: 582, Height: 155 / X: 843; Y: 450


labelImg
OK
Cancel
Millennium Falcon
Millennium Falcon
Tie Fighter


Millennium Falcon
Tie Fighter
Tie Fighter
檔案清單
C:\AIoT\ch10\ch10-4-2\frames\frame60

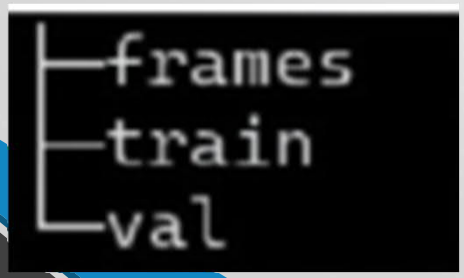
➢ 3.分割成訓練和驗證資料集 Step3_dataset_splitter.py

Step3_dataset_splitter.py

```python
import os
import shutil
import random

input_dir = "frames"  # 請將此處替換為您的輸入目錄
train_dir = "train"   # 訓練資料集目錄
val_dir = "val"       # 驗證資料集目錄
split_ratio = 0.8     # 訓練與驗證的分割比例
img_type = ".jpg"     # 圖檔類型
operation = "copy"    # 選擇 "move" 搬移檔案 或 "copy" 複製
# ----------------------------------------
```

```
>>> %Run Step3_dataset_splitter.py
使用 copy 操作處理 frames 目錄的資料集
資料集分割成 train 和 val 目錄的資料集
使用的分割比例: 0.8
```



➢ 4.將資料集再分割成圖檔和標註檔資料夾 Step4_dataset_organizer.py

Step4_dataset_organizer.py

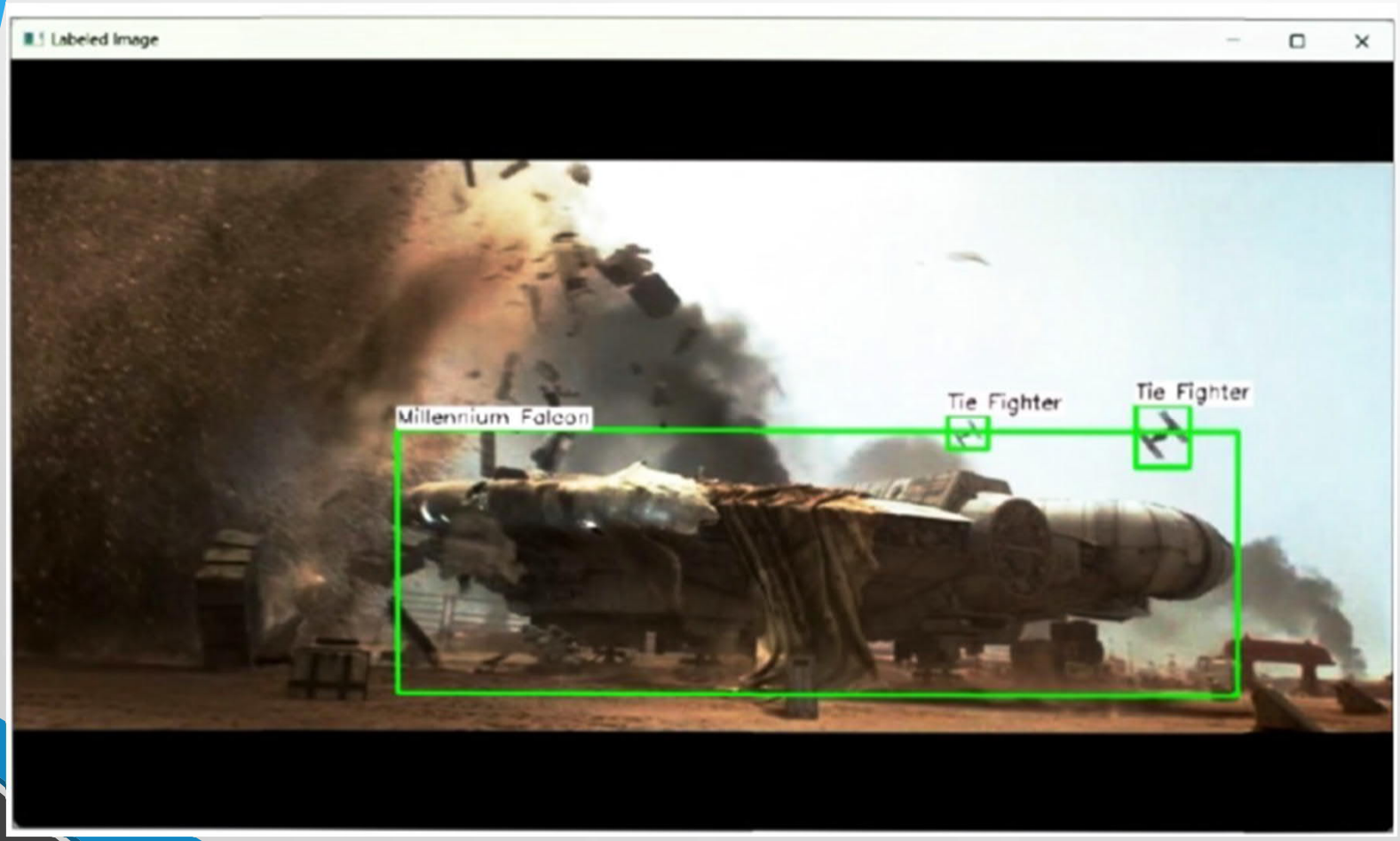
```python
import os
import shutil

train_dir = "train"    # 訓練資料集目錄
val_dir = "val"        # 驗證資料集目錄
# ----------------------------------------
```



- 5.瀏覽Labellmg 標註的圖檔

Step5_annotation_image_viewer.py



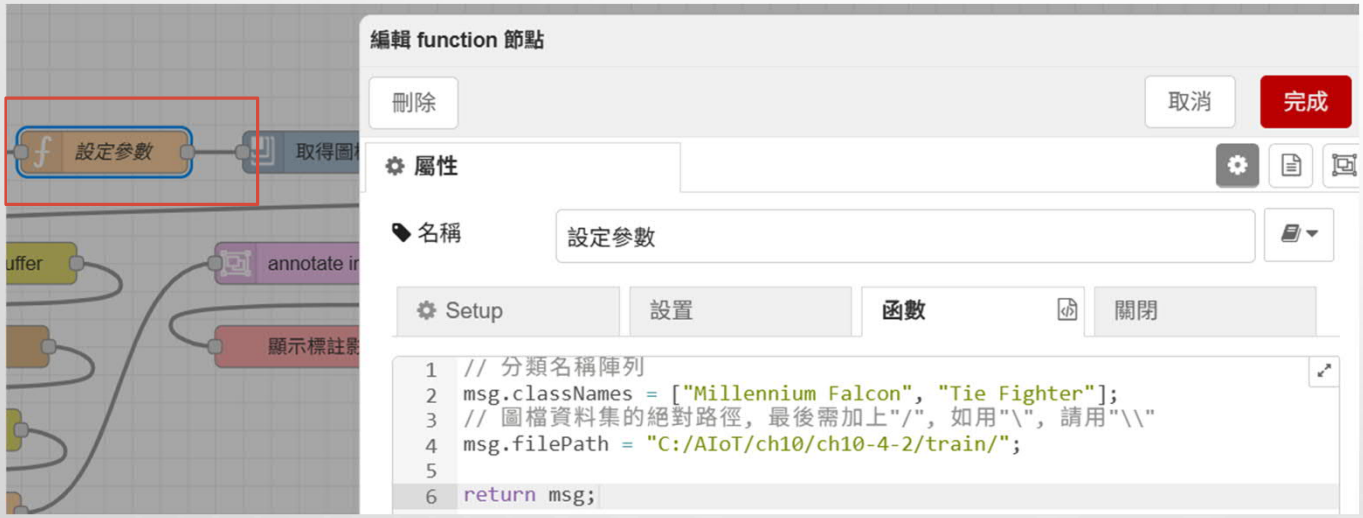
4. 在Node-RED 顯示標註圖檔

- 1.安裝節點:

node-red-contrib-image-info



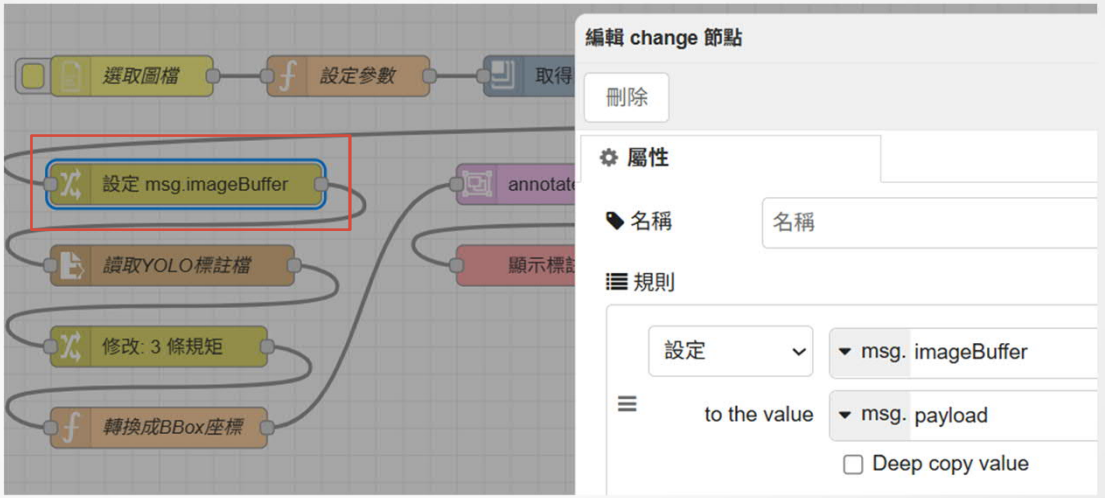


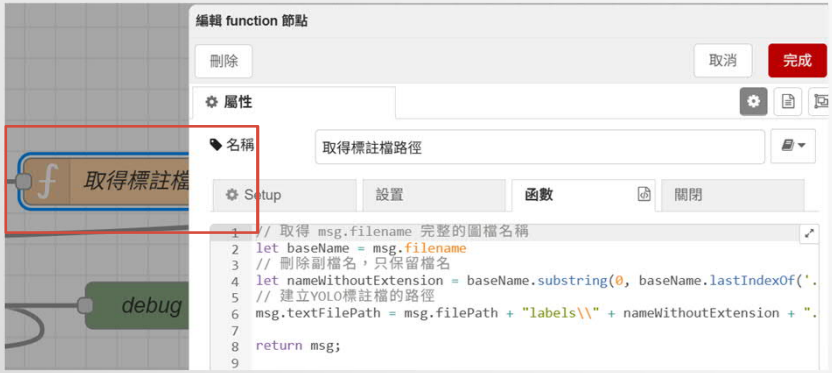



```
// 分類名稱陣列
msg.classNames = ["Millennium Falcon", "Tie Fighter"];
// 圖檔資料集的絕對路徑, 最後需加上"/", 如用"\", 請用"\\"
msg.filePath = "C:/AIoT/ch10/ch10-4-2/train/";
return msg;
```

換到你們train資料夾位置

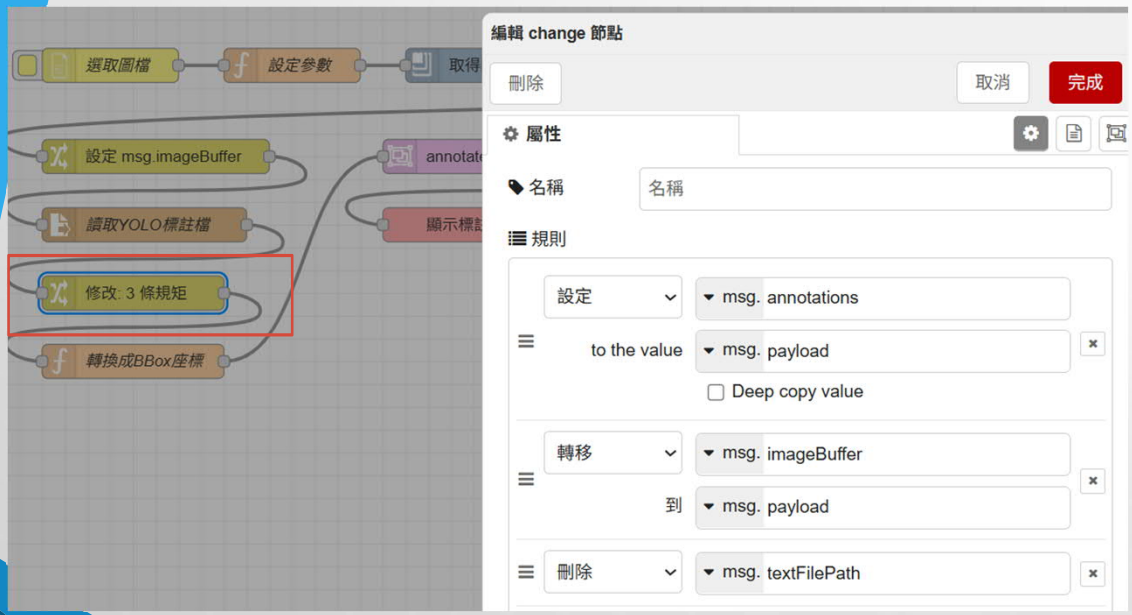


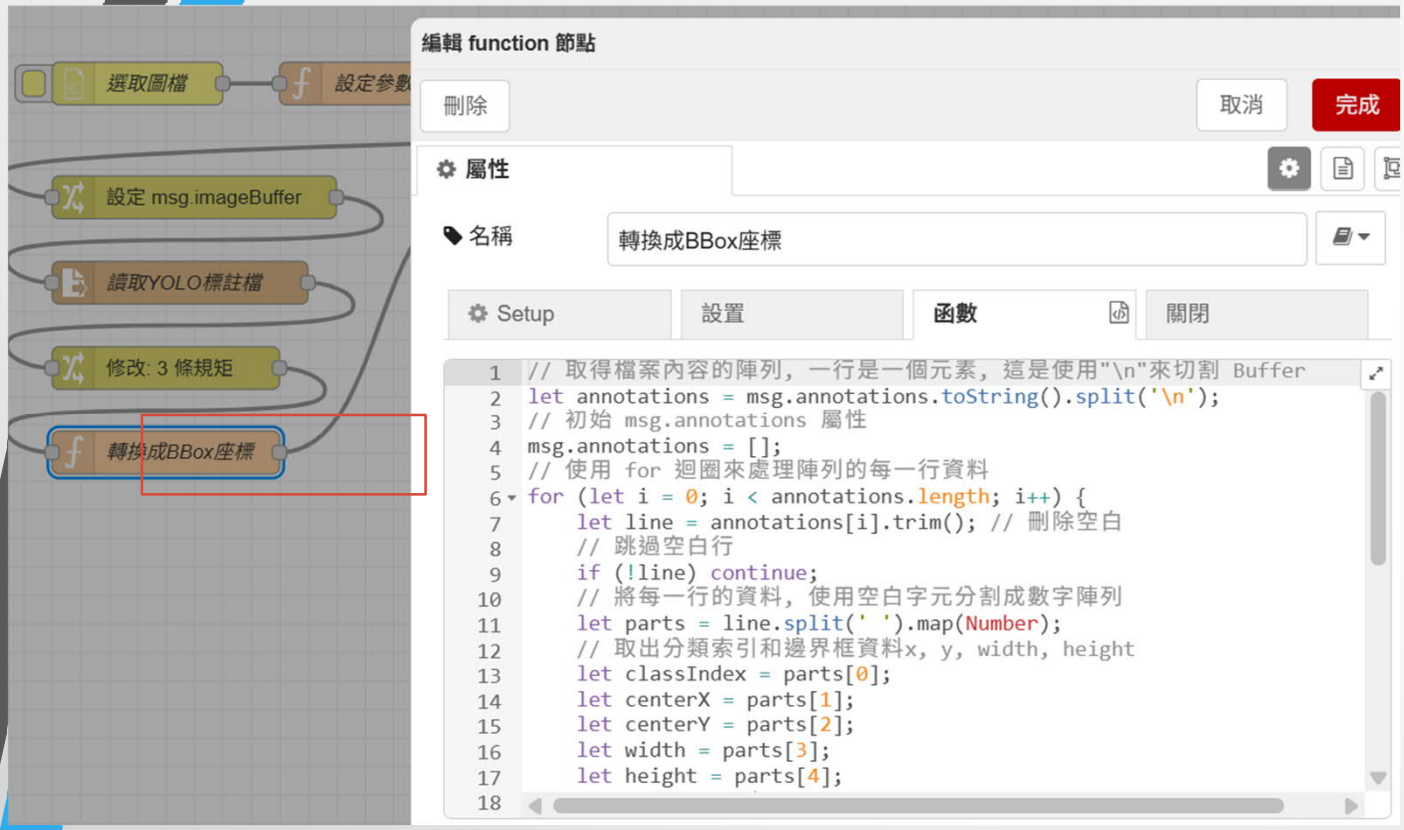





```
// 取得 msg.filename 完整的圖檔名稱
let baseName = msg.filename
// 刪除副檔名，只保留檔名
let nameWithoutExtension = baseName.substring(0, baseName.lastIndexOf('.'));
// 建立YOLO標註檔的路徑
msg.textFilePath = msg.filePath + "labels\\" + nameWithoutExtension + ".txt";
return msg;
```

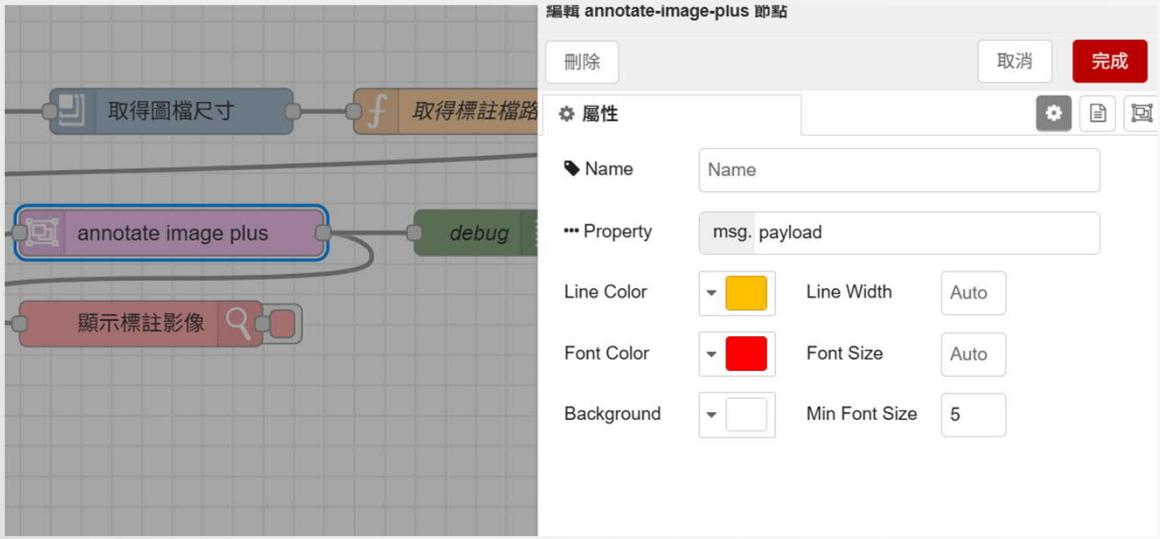

選取圖檔
設定參數
設定 msg.imageBuffer
讀取YOLO標註檔
修改: 3 條規矩
轉換成BBox座標
編輯 read file 節點
刪除
取消
完成
屬性
檔案名
msg. textFilePath
輸出
一個Buffer物件
名稱
讀取YOLO標註檔
提示: 檔案名應該是絕對路徑，否則它將相對於Node-RED進程的工作目錄。

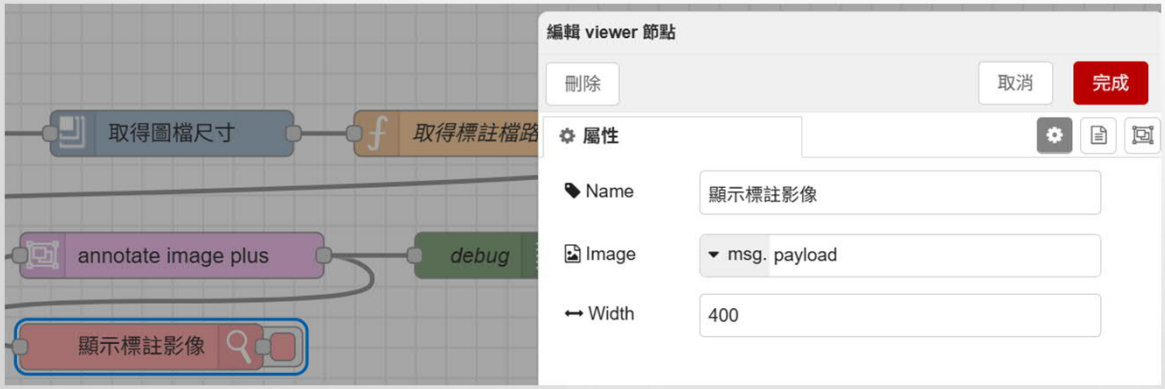

選取圖檔
設定參數
設定 msg.imageBuffer
讀取YOLO標註檔
修改: 3 條規矩
轉換成BBox座標
編輯 change 節點
刪除
取消
完成
屬性
名稱
名稱
規則
設定
msg. annotations
to the value
msg. payload
Deep copy value
轉移
msg. imageBuffer
到
msg. payload
刪除
msg. textFilePath




```
// 取得檔案內容的陣列, 一行是一個元素, 這是使用"\n"來切割 Buffer
let annotations = msg.annotations.toString().split('\n');
// 初始 msg.annotations 屬性
msg.annotations = [];
// 使用 for 迴圈來處理陣列的每一行資料
for (let i = 0; i < annotations.length; i++) {
  let line = annotations[i].trim(); // 刪除空白
  // 跳過空白行
  if (!line) continue;
  // 將每一行的資料, 使用空白字元分割成數字陣列
  let parts = line.split(' ').map(Number);
  // 取出分類索引和邊界框資料x, y, width, height
  let classIndex = parts[0];
  let centerX = parts[1];
  let centerY = parts[2];
  let width = parts[3];
  let height = parts[4];
  // console.log("YOLO:", classIndex, centerX, centerY, width, height);
  // 將 YOLO 格式轉換為 bbox 格式 (x, y, width, height)
  let bbox_x = (centerX - width / 2) * msg.width;
  let bbox_y = (centerY - height / 2) * msg.height;
  let bbox_width = width * msg.width;
  let bbox_height = height * msg.height;
  // console.log("BBOX:", bbox_x, bbox_y, bbox_width, bbox_height);
  // 建立每一筆標註資料的 JSON 物件
  let jsondata = {
    label: msg.classNames[classIndex], // 根據索引取得分類名稱
    bbox: [bbox_x, bbox_y, bbox_width, bbox_height],
    labelLocation: "top" // 預設標籤位置為 "top"
  }
  // console.log(jsondata)
  // 新增一個標註物件
  msg.annotations.push(jsondata);
}

return msg;
```

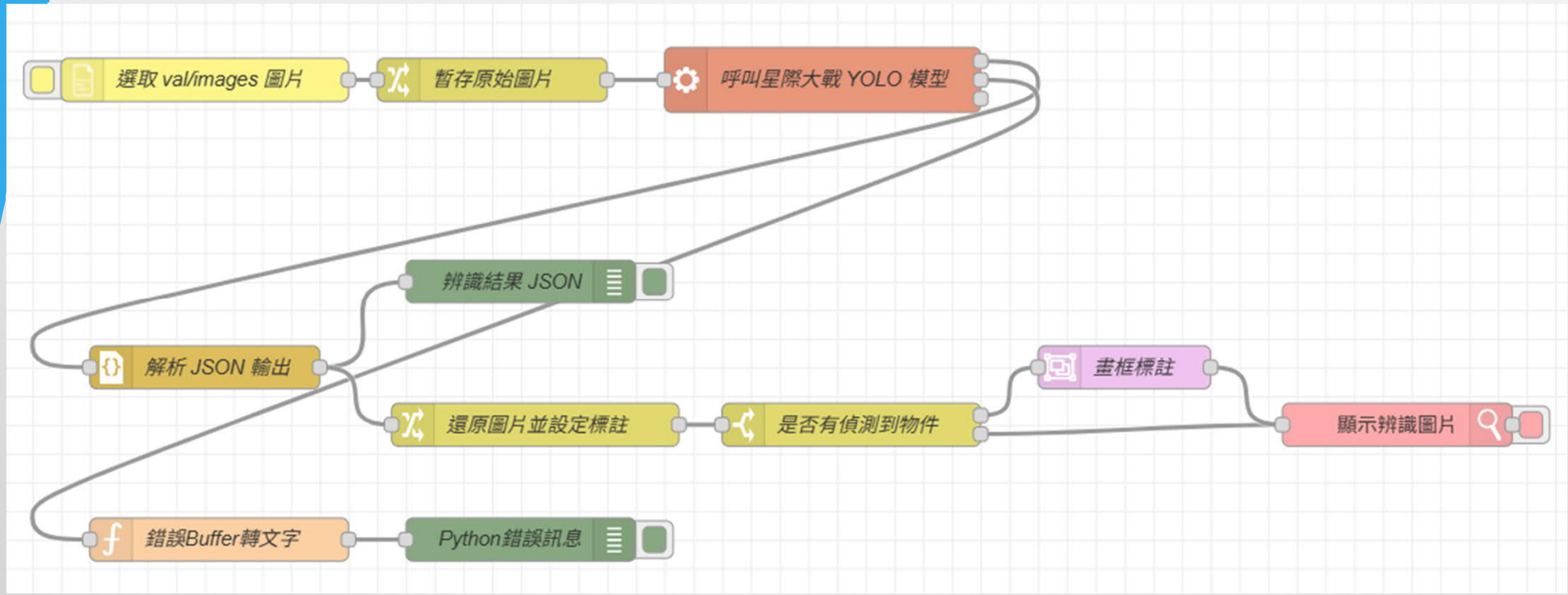

取得圖檔尺寸
取得標註檔路
annotate image plus
debug
顯示標註影像
編輯 annotate-image-plus 節點
刪除
取消
完成
屬性
Name
Name
Property
msg. payload
Line Color
Line Width
Auto
Font Color
Font Size
Auto
Background
Min Font Size
5


取得圖檔尺寸
取得標註檔路
annotate image plus
debug
顯示標註影像
編輯 viewer 節點
刪除
取消
完成
屬性
Name
顯示標註影像
Image
msg. payload
Width
400

資料夾 train\images



5. 建立YOLO 訓練及使用



1.Run
001_create_data_yaml.py
安裝 ultralytics



2.Run
002_train_starwars.py





